Studio Nx

JS 296 — 화면과 오디오 이어 주기

2026-10-19

JavaScript 296 — 화면과 오디오 이어 주기 (`sequencer-audio.js` 외)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-audio.js` (신규), `sequencer-arrange.js`, `sequencer-devices.js`, `sequencer.js` (연결 부분만 수정)
작업	S1-11a 재생 오디오 그래프: 화면의 값을 읽어 오디오 세션에 넘기고, 재생 버튼을 소리에 연결 (FR-201, FR-205)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 295 레벨 미터, 다음 문서 없음 (작업 개요 82)

1. 하는 일

오디오 폴더(`audio/`)는 화면을 모르는 순수한 소리 코드입니다. 이 파일이 둘 사이를 잇습니다. 화면의 클립, 트랙, BPM, 루프를 읽어서 세션에 넘기고, ▶ ⏸ ■ 버튼이 실제 소리를 내게 합니다.

```
화면 (DOM, 서버가 준 데이터)                          소리 (audio/)
  .sq-clip--audio ┐
  .sq-clip--midi  ┤  sequencer-audio.js          ┌─▶ PlaybackSession
  트랙, 악기, 이펙트 ┼─ collect... / playback... ──▶ │     ├─ 믹서
  #bpm, Loop      ┘   (읽기만 함)                 │     ├─ 클립, 노트 재생
                                                  │     └─ 이펙트 체인
  ▶ ⏸ ■ 버튼 ─▶ setTransportHooks (sequencer-arrange.js) ─▶ onPlay / onPause
  미터 막대 ◀─ setMeterSource (sequencer-devices.js) ◀── session.meterLevels
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer.js`	모듈을 불러온 뒤 `initAudio()`를 한 번 부릅니다.
재생 버튼	편곡의 `play()`가 `onPlay(bar)`를 부르면 세션이 그 마디부터 소리를 시작합니다.
일시정지, 정지	편곡의 `pause()`가 `onPause()`를 부르면 소리를 끊습니다.
헤드리스 시험	`audioSession()`으로 세션을 얻어 마스터 출력의 피크를 잽니다.

3. 코드

3.1 화면에서 클립 모으기

```
function collectAudioClips() {
  const list = [];
  Array.prototype.forEach.call(document.querySelectorAll('.sq-clip--audio'), function (el) {
    const trackRow = el.closest('.sq-track');
    const trackId = trackRow ? Number(trackRow.dataset.trackId) : NaN;
    const sampleId = el.dataset.sampleId ? Number(el.dataset.sampleId) : NaN;
    if (!isFinite(trackId) || !isFinite(sampleId)) return;
```

```
    const rec = el.dataset.clipId ? clipRecord(el.dataset.clipId) : null;
    const startBar = parseFloat(el.style.getPropertyValue('--s')) || 0;
    const lengthBar = parseFloat(el.style.getPropertyValue('--l')) || 0;
    const offsetBar = parseFloat(el.style.getPropertyValue('--o')) || 0;
    if (lengthBar <= 0) return;
    list.push({
      clip_id: el.dataset.clipId ? Number(el.dataset.clipId) : null,
      track_id: trackId,
      sample_id: sampleId,
      start_tick: Math.round(startBar * TICKS_PER_BAR),
      length_tick: Math.round(lengthBar * TICKS_PER_BAR),
      offset_tick: Math.round(offsetBar * TICKS_PER_BAR),
      gain_db: rec ? Number(rec.gain_db) || 0 : 0,
      is_looped: rec ? !!rec.is_looped : false,
      pitch_semitones: rec ? Number(rec.pitch_semitones) || 0 : 0,
    });
  });
  return list;
}
```

```
function collectMidiClips() {
  const list = [];
  Array.prototype.forEach.call(document.querySelectorAll('.sq-clip--midi'), function (el) {
    const trackRow = el.closest('.sq-track');
    const trackId = trackRow ? Number(trackRow.dataset.trackId) : NaN;
    if (!isFinite(trackId)) return;
    const startBar = parseFloat(el.style.getPropertyValue('--s')) || 0;
    const lengthBar = parseFloat(el.style.getPropertyValue('--l')) || 0;
    const offsetBar = parseFloat(el.style.getPropertyValue('--o')) || 0;
    if (lengthBar <= 0) return;
    const notes = el.__notes ? el.__notes : (el.dataset.patternId ? patternNotes(el.dataset.patternId) :
[]);
    list.push({
      track_id: trackId,
      start_tick: Math.round(startBar * TICKS_PER_BAR),
      length_tick: Math.round(lengthBar * TICKS_PER_BAR),
      offset_tick: Math.round(offsetBar * TICKS_PER_BAR),
      notes: notes,
    });
  });
  return list;
}
```

3.2 세션 만들기와 연결

```
function getSession() {
  if (session) return session;
  session = new PlaybackSession({
    getTracks: playbackTracks,
    getClips: collectAudioClips,
    getMidiClips: collectMidiClips,
    getInstruments: playbackInstruments,
    getEffects: playbackEffects,
    getBpm: currentBpm,
    getLoop: function () {
```

```
      const loop = getLoop();
      if (!loop.on || !(loop.length > 0)) return null;
      return { start: Math.round(loop.start * TICKS_PER_BAR), end: Math.round((loop.start + loop.length)
* TICKS_PER_BAR) };
    },
    loadSampleBytes: loadSampleBytes,
  });
  return session;
}
```

```
export function initAudio() {
  setTransportHooks({
    onPlay: function (bar) {
      const s = getSession();
      startMeters();
      s.start(Math.round(bar * TICKS_PER_BAR)).then(startRefresh).catch(function (err) {
        if (window.console) console.warn('[audio] 재생을 시작하지 못했습니다', err);
      });
    },
    onPause: function () {
      stopRefresh();
      if (session) session.pause();
    },
  });
  // 미터 막대가 진짜 소리 크기를 읽게 함 (세션이 아직 없으면 null → 막대는 0 으로 내려감)
  setMeterSource(function (trackId) { return session ? session.meterLevels(trackId) : null; });
  if (window.console) console.debug('[audio] connected, PPQ', PPQ);
}
```

3.3 편곡 쪽 연결 고리 (`sequencer-arrange.js`)

```
export function setTransportHooks(hooks) {
  transportHooks.onPlay = hooks && hooks.onPlay ? hooks.onPlay : null;
  transportHooks.onPause = hooks && hooks.onPause ? hooks.onPause : null;
}
```

3.4 서버 데이터를 재생용으로 바꾸는 읽기 함수 (`sequencer-devices.js`)

```
export function playbackTracks() {
  return tracks.map(function (t) {
    const row = document.querySelector('.sq-track[data-track-id="' + t.track_id + '"]');
    const mute = row ? row.querySelector('.is-mute') : null;
    const solo = row ? row.querySelector('.is-solo') : null;
    return Object.assign({}, t, {
      is_muted: mute ? mute.getAttribute('aria-pressed') === 'true' : !!t.is_muted,
      is_solo: solo ? solo.getAttribute('aria-pressed') === 'true' : !!t.is_solo,
    });
  });
}
```

```
export function playbackInstruments(trackId) {
  const d = devices[trackId];
  if (!d) return [];
  return (d.instruments || []).map(function (inst) {
```

```
    const plugin = pluginById(inst.plugin_id);
    return {
      id: inst.track_instrument_id,
      plugin_code: plugin ? plugin.code : '',
      schema: plugin && plugin.param_schema ? plugin.param_schema : [],
      params: inst.params || null,
      volume_db: Number(inst.volume_db) || 0,
      pan: Number(inst.pan) || 0,
      is_enabled: inst.is_enabled !== false,
    };
  });
}
```

```
export function playbackEffects(trackId) {
  const d = devices[trackId];
  if (!d) return [];
  return (d.effects || []).map(function (fx) {
    const plugin = pluginById(fx.plugin_id);
    return {
      id: fx.track_plugin_id,
      plugin_code: plugin ? plugin.code : '',
      schema: plugin && plugin.param_schema ? plugin.param_schema : [],
      params: fx.params || {},
      is_bypassed: !!fx.is_bypassed,
      slot_index: fx.slot_index,
    };
  });
}
```

```
export function patternNotes(patternId) {
  const p = patternsById[patternId];
  return p && p.notes ? p.notes : [];
}
```

```
export function clipRecord(clipId) {
  const found = clips.filter(function (c) { return String(c.clip_id) === String(clipId); })[0];
  return found || null;
}
```

4. 최신 문법으로 쓰면

```
// playbackInstruments 를 최신 문법으로 쓴 모양
export const playbackInstruments = (trackId) =>
  (devices[trackId]?.instruments ?? []).map((inst) => {
    const plugin = pluginById(inst.plugin_id);
    return {
      id: inst.track_instrument_id,
      plugin_code: plugin?.code ?? '',
      schema: plugin?.param_schema ?? [],
      params: inst.params ?? null,
      volume_db: Number(inst.volume_db) || 0,
      pan: Number(inst.pan) || 0,
      is_enabled: inst.is_enabled !== false,
```

```
    };
  });
```

전통 문법 (실제 코드)	최신 문법
`if (!d) return []; d.instruments || []`	`devices[trackId]?.instruments ?? []`
`plugin ? plugin.code : ''`	`plugin?.code ?? ''`
`function (inst) { ... }`	`(inst) => { ... }`
`Array.prototype.forEach.call(NodeList, ...)`	`[...NodeList].forEach` 또는 `for...of`

5. 왜 이렇게

선택	이유
화면을 읽는 코드를 한 파일에 모았습니다	`audio/` 는 화면(DOM)을 전혀 몰라야 Node에서 시험하고 WAV 출력에서도 쓸 수 있습니다. 화면과 닿는 부분을 이 파일 하나로 제한했습니다.
클립 위치와 길이는 화면 값을, 게인과 루프와 음정은 서버 기록을 읽습니다	사용자가 끌어서 옮긴 값은 화면이 가장 최신입니다. 화면에 없는 속성은 서버 기록에 있습니다.
재생 시작 때 스냅샷을 만듭니다	재생 중에 편집해도 이미 예약한 소리와 어긋나지 않습니다.
저장되지 않은 트랙과 샘플이 없는 클립은 건너뜁니다	서버 번호(숫자)가 없는 임시 요소는 소리를 낼 재료가 없습니다.
뮤트와 솔로는 화면 버튼 상태를 따릅니다 (`playbackTracks`)	버튼을 누르면 화면이 먼저 바뀌고 저장은 뒤따릅니다. 소리가 저장을 기다리면 반응이 늦게 느껴집니다.
피아노롤에서 고친 노트는 `__notes`를, 아니면 서버 패턴의 노트를 씁니다	피아노롤을 연 클립은 화면이 더 최신이고, 열지 않은 클립은 서버가 준 값이 맞습니다.
재생 연결을 훅(`setTransportHooks`)으로 합니다	편곡 파일이 오디오 파일을 직접 가져오면 서로 가져오는 순환이 생깁니다. 편곡은 "재생이 시작됨"만 알립니다.
샘플 내려받기에 실패해도 재생선은 움직입니다	소리 시작은 `Promise`로 처리하고 실패하면 경고만 남깁니다. 화면 재생은 소리와 독립적으로 동작합니다.
믹서 갱신 타이머를 재생 중에만 돌립니다	150ms마다 볼륨, 뮤트, 이펙트를 다시 읽습니다. 멈춰 있을 때는 돌릴 필요가 없습니다.

6. 시험

- `docs/handoff/s1-11a_tests/ui.mjs`, `ui_midi.mjs`, `ui_fx.mjs`, `ui_meter.mjs`가 모두 이 연결을 통해 실제 화면에서 동작합니다. 사인파 클립을 만들고 ▶를 눌러 마스터 출력의 피크를 읽는 방식입니다.
- 시험 설계에서 배운 점: 예시 화면의 재생선이 처음부터 0마디가 아니고, 기존 트랙에 이미 이펙트가 있고 볼륨이 낮을 수 있습니다. 그래서 시험은 늘 전용 트랙과 새 Synth를 만들고, 시작 전에 ■로 재생선을 0으로 되돌리며, 끝나면 모두 지웁니다.